

ARO: Autoregressive Operator Learning for Transferable and Multi-fidelity 3D-IC Thermal Analysis with Active Learning

Mingyue Wang^{1,2,#}, Yuanqing Cheng^{1,2,#}, Weiheng Zeng¹, Zhenjie Lu³, Vasilis F. Pavlidis⁴, Wei W. Xing^{5*}

¹ School of Integrated Circuit Science and Engineering, Beihang University, Beijing, China

² National Key Laboratory of Spintronics, Beihang University, Hangzhou, China

³ School of Mechanical, Electrical and Control Engineering, Shenzhen University, Shenzhen, China

⁴ ECE Department, Aristotle University of Thessaloniki, Thessaloniki, Greece. ⁵ SoMas, The University of Sheffield, U.K.

{Mia_wang,yuanqing,Harryzeng}@buaa.edu.cn,luzhenjie2020@szu.edu.cn,vpavlid@ece.auth.gr,w.xing@sheffield.ac.uk

ABSTRACT

As 3D integrated circuits (ICs) have emerged as a promising direction in the semiconductor industry, thermal issues in 3D-ICs have become increasingly prominent. In this work, we develop a novel machine learning (ML) thermal analysis framework, namely Autoregressive Operator (ARO), to address the pressing need for rapid yet highly accurate thermal predictions during the chip design process. Unlike traditional ML-based methods that can only deal with scenarios of well-defined input-output domains, ARO learns the thermal diffusion operator such that it can generalize to any unseen circuits and map the power traces to the steady-state/transient thermal spatial-temporal distributions. To further reduce the computational demand of data preparation, we equip ARO with multi-fidelity fusion to exploit the advantage of computationally cheap low-fidelity simulations and expensive high-fidelity simulations and active learning to guide the preparation of training data. Our results show that, for the unseen testing cases, a well-trained ARO can produce accurate results with about 1000× speedup compared to MTA. Moreover, equipped with active learning, ARO achieves at least 25% data reduction compared to pseudo-random strategies.

KEYWORDS

3D-IC, Thermal Analysis, Fourier Neural Operator, Multi-fidelity

1 INTRODUCTION

The ever-expanding growth of modern IC technology has driven functionality and performance improvements in IC design, primarily through increasing packaging density and clock frequency. This trend of high-density integration on-chip has intensified thermal dissipation challenges, particularly in the context of 3D chip stacking, which has emerged as a promising solution for addressing the limitations of conventional 2D and 2.5D ICs. However, 3D chips generally suffer from high temperatures and hot spot issues, which can undermine the reliability of both interconnects and transistors [1]. Moreover, thermal gradients across the chip can precipitate functional or timing failures. Thus, thermal analysis is a critical step in the IC design flow to ensure reliability, necessitating rapid thermal

simulation and analysis tools, especially for optimization-related applications, such as thermal-aware floorplan and placement [2].

The thermal behavior of a chip can be well characterized by a 3D heat conduction partial differential equation (PDE), which is typically solved using Finite Element Analysis (FEA) [3] for challenging problems with complex geometries. FEA solvers are widely employed in commercial tools like COMSOL and ANSYS, while Finite Difference Method (FDM) is commonly implemented in open-source solvers [4]. Although these simulators yield accurate temperature estimations, they demand substantial computing resources, which can be prohibitive where optimization is also involved.

With the rapid development of machine learning (ML) methods across various fields showcasing exceptional modeling capabilities, researchers have made important strides in integrating ML with the 3D-IC design process to replace or accelerate traditional thermal solvers [5–8]. For instance, [5] proposed a fast full-chip thermal numerical analysis method using the enhanced Physical Information Neural Network (PINN) framework, called Thermal-PINN to emulate the mapping from initial temperature, thermal conductivity, to thermal distribution. However, this work does not delve into the influence of power distribution or extend beyond 2D-ICs. Thermal-GAN leverages Generative Adversarial Networks (GAN) to achieve real-time prediction of chip thermal distribution using performance metrics from monitors [6]. This approach has been expanded by integrating an attention mechanism to further improve accuracy [7]. However, these studies solely focus on real-time chip temperature prediction with a well-defined input space, hindering their practical application where the input space cannot be easily pre-defined.

To resolve this problem, DeepOHeat [8] integrated operator learning (OL) and PINN, focusing on learning the mapping from power map, heat transfer coefficient to thermal distribution in 3D-ICs. Despite its success in predictions with a varying input space, it only considers a simple steady-state problem (maximum 21-by-21 resolution fields vs. 151-by-151 in our work). Thus, it cannot generalize well to a wider spectrum of high-resolution applications to denote a detailed thermal field. Moreover, it requires a large amount of pre-computed high-fidelity data, which is expensive to generate using general thermal simulation tools.

Inspired by the fact that thermal simulations can be run at low-fidelity to provide training data at reduced computational cost, GAR [9] proposed a universal framework based on Gaussian Processes (GP) for multi-fidelity fusion of high-dimensional fields. However, its reliance on GP makes it difficult to deal with a function input

*# Authors contributed equally to this work.

* Corresponding author.

This work is supported in part by Beijing Municipal Natural Science Foundation No. Z230004, National Natural Science Foundation of China under Grant No. 92373205, Shenzhen Science and Technology Program under Grant No. SGD20230116093303006 and KJZD20231023100201003, the Strategic Priority Research Program of Chinese Academy of Sciences under Grant No. XDB44000000 and the Fundamental Research Funds for the Central Universities.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from Permissions@acm.org.

ICCAD '24, October 27–31, 2024, New York, NY, USA

© 2024 Copyright is held by the owner/author(s). Publication rights licensed to ACM.

ACM 979-8-4007-1077-3/24/10...

<https://doi.org/10.1145/3676536.3676713>

space, and it may not be well-suited for thermal distribution prediction in 3D-ICs. A multi-fidelity deep learning approach that leverages low-fidelity data to enhance high-fidelity models has been proposed [10]. Lyu *et al.* [11] introduced Fourier Neural Operator (FNO) with multi-fidelity fusion, harnessing FNO's inherent resolution-invariant capability to easily adapt to different resolution scenarios for the inputs. Due to the deterministic nature of deep learning models, most of these methods still require extensive training data, which is hard to obtain for our 3D-IC thermal analysis. Recent work by Li *et al.* [12] introduced MRA-FNO, which incorporated active learning to guide the selection of valuable samples for multi-fidelity models, significantly reducing the computational cost of data preparation. However, MRA-FNO is designed for 2D or 3D problems and scales poorly to 4D (three space dimensions and time dimension) transient thermal analysis.

For 3D-IC thermal analysis, the existing ML-based methods are limited in their ability to generalize to unseen circuits and require a large amount of high-fidelity data. To this end, we propose Autoregressive Operator learning (ARO), a novel framework that combines the latest advancements in AI for physical field emulation, i.e., FNO and Generalized Autoregressive model (GAR), to learn the thermal diffusion operator with multi-fidelity simulation results. Moreover, we equip ARO with active learning to guide the preparation of training data, which significantly reduces the computational cost of data preparation. The novelty of this work includes:

- A novel autoregression operator learning framework, ARO, is proposed to learn the implicit thermal diffusion operator with multi-fidelity simulation data, demonstrating a data reduction of at least 23% compared to traditional FNOs while providing similar prediction accuracy.
- ARO is equipped with active learning to guide the preparation of training data, further improves the RMSE performance with a minimum 25% data reduction compared to random choosing strategies.
- Compared to conventional simulation-based thermal analysis, ARO achieves high accuracy (a coefficient of determination over 0.98) with a 1000× speedup.

The rest of the paper is organized as follows: Section 2 introduces the background of ARO. Section 3 delineates proposed approaches to enabling ARO. Section 4 describes the experiment results and Section 5 draws our conclusion.

2 BACKGROUND

2.1 Thermal Modeling of 3D Integrated Circuits

3D-IC thermal analysis focuses on simulating the heat transfer process in 3D integrated circuits. Utilizing thermal analysis tools, the temperature fluctuations are determined at each location on the chip over time, given the power traces of each component and other key physical information of the system, e.g., chip layout and stacking geometry. Fig. 1 illustrates the workflow of 3D-IC thermal analysis.

Defining $E \subset \mathbb{R}^3$ as the IC domain, and $\mathbf{r} \in E$ as the location of a point on the chip, the governing equation in IC thermal analysis, serving as thermal diffusion equation, can be expressed as follows:

$$c(\mathbf{r}) \frac{\partial T(\mathbf{r}, t)}{\partial t} = p(\mathbf{r}, t) + \nabla \cdot (\kappa(\mathbf{r}) \nabla T(\mathbf{r}, t)), \quad (1)$$

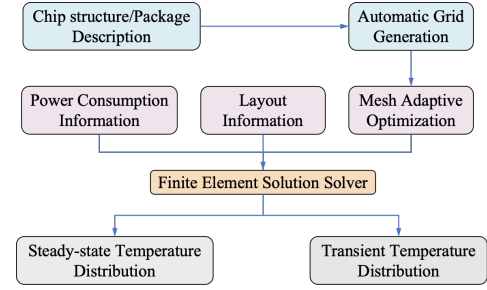


Figure 1: The workflow of 3D-IC thermal analysis.

where $c(\mathbf{r})$, $\kappa(\mathbf{r})$ are the heat capacity and the thermal conductivity at the point $\mathbf{r}$ respectively. $T(\mathbf{r}, t)$, $p(\mathbf{r}, t)$ are the temperature and power density at the point $\mathbf{r}$ and time t respectively. Assume a homogeneous initial condition at $t = 0$ for the IC, i.e.,

$$T(\mathbf{r}, 0) = T_0, \quad (2)$$

The boundary condition is defined as:

$$\kappa(\mathbf{r}) \frac{\partial T(\mathbf{r}, t)}{\partial \mathbf{n}} = \begin{cases} p_0(\mathbf{r}, t) & \text{on } (\partial E)_1 \\ \eta(T_0 - T(\mathbf{r}, t)) & \text{on } (\partial E)_2 \end{cases}, \quad (3)$$

where $\frac{\partial T(\mathbf{r}, t)}{\partial \mathbf{n}}$ is the derivative of temperature along the normal vector of the chip surface. $p_0(\mathbf{r}, t)$ represents the power density, and η represents the thermal transmissivity. Boundaries around the chip are represented as $(\partial E)_1$, while the top and bottom surfaces are denoted as $(\partial E)_2$. The thermal analysis utilizes FEA to solve Eq. (1) and obtain the temperature distribution $T(\mathbf{r}, t)$.

2.2 Problem Definition

As is shown in Fig. 1, with the chip structure and layout information predetermined, we consider $p(\mathbf{r}, t)$ as the input of thermal analysis, aiming to derive the output $T(\mathbf{r}, t)$ in Eq. (1).

Operator learning provides a new perspective for thermal analysis distinguished from traditional numerical methods like FEA, as it does not directly solve Eq. (1) but seeks the internal connection between inputs and outputs through massive observations. In operator learning, we focus on the non-linear mapping:

$$G^* : \mathcal{P}(D; \mathbb{R}) \rightarrow \mathcal{T}(D; \mathbb{R}), \quad (4)$$

$$p \mapsto T,$$

where $D = E \times [0, t_{max}]$, $\mathcal{P}(D; \mathbb{R})$ is a function space, whose element p is a function mapping $(\mathbf{r}, t) \in D$ to power density $p(\mathbf{r}, t) \in \mathbb{R}$; $\mathcal{T}(D; \mathbb{R})$ is also a function space, whose element T is a function mapping $(\mathbf{r}, t) \in D$ to temperature $T(\mathbf{r}, t) \in \mathbb{R}$.

Our goal is to approximate G^* in Eq. (4) by using datasets of observations $\mathcal{D} = \{p_j(\mathbf{r}, t), T_j(\mathbf{r}, t)\}_{j=1}^K$, where K is the size of dataset. Among methods of operator learning, Fourier neural operator perfectly fits with our problem, for it is fast to evaluate through training, and it is resolution-invariant, compensating for the speed-accuracy trade-off found in traditional numerical methods.

2.3 Fourier Neural Operator

Mainstream neural network research has traditionally focused on the challenge of learning the mappings within finite-dimensional Euclidean spaces. Recent research has broadened the scope to include the exploration of neural operators capable of learning mappings within function spaces. A notable advancement in this field was pioneered by [13], which introduced a novel class of neural

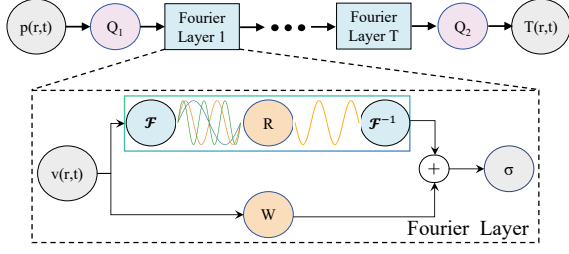


Figure 2: The architecture of the Fourier neural operator.

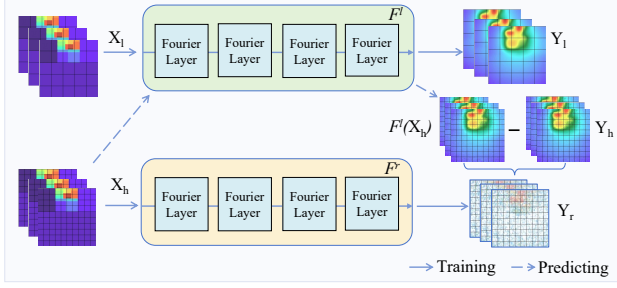


Figure 3: The general framework of ARO employing multi-fidelity fusion.

operators known as the Fourier neural operator (FNO). The operator framework is designed to excel in the task of mapping initial conditions to solutions for PDEs.

Specifically, FNO is constructed by stacking Fourier layers, as shown in Fig. 2. Assuming $\{v_n\}_{n=0}^N$ be a series of functions mapping D to $\mathbb{R}^d$, the workflow of FNO can be mainly considered as an iterative scheme $\{v_n \mapsto v_{n+1}\}_{n=0}^{N-1}$. During this process, the input $p(\mathbf{r}, t)$ is first transformed to $v_0(\mathbf{r}, t) = Q_1(p(\mathbf{r}, t))$ by a fully-connected layer $Q_1 : \mathbb{R} \rightarrow \mathbb{R}^d$ ($d > 1$). The $v_0(\mathbf{r}, t)$ goes through several iterations by

$$v_{n+1}(\mathbf{r}, t) = \sigma(Wv_n(\mathbf{r}, t) + \mathcal{F}^{-1}(R \cdot \mathcal{F}v_n)(\mathbf{r}, t)), \quad (5)$$

where $W \in \mathcal{L}(\mathbb{R}^d; \mathbb{R}^d)$ is a linear transformation on $\mathbb{R}^d$. $\mathcal{F}$ is the Fourier transform of a function and $\mathcal{F}^{-1}$ is its inverse transform. R is the Fourier transform on the lower Fourier modes, while $\sigma : \mathbb{R}^d \rightarrow \mathbb{R}^d$ represents a non-linear activation function. After K iterations, $v_n(\mathbf{r}, t)$ is transferred to the output $T(\mathbf{r}, t)$ by a layer $Q_2 : \mathbb{R}^d \rightarrow \mathbb{R}$ and $T(\mathbf{r}, t) = Q_2(v_n(\mathbf{r}, t))$. The fundamental feature of FNO is its utilization of the Fourier layer, which can accommodate functions discretized in any manner. This is enabled by learning parameters directly in Fourier space, implying that analyzing a function in physical space is akin to projecting it onto a defined wave function across space. Such a property renders FNO resolution-invariant and capable of adjusting to diverse input and output resolutions seamlessly, without necessitating alterations to the model.

3 PROPOSED APPROACH

3.1 Multi-fidelity Enabled FNO - ARO

While neural operator frameworks excel at modeling intricate phenomena, they typically demand substantial volumes of data for effective training, which is frequently unavailable or prohibitively expensive. This challenge can be mitigated through the implementation of multi-fidelity operator learning [14], a strategy that involves training the model using abundant, cost-effective low-fidelity data

alongside limited, expensive high-fidelity data. In thermal analysis, low-fidelity data is acquired through sparse meshing, while high-fidelity data is obtained via fine meshing of the spatial domain. We denote the low-fidelity data as $\mathcal{D}_l = \{X_l, Y_l\}$ and high-fidelity data as $\mathcal{D}_h = \{X_h, Y_h\}$. Here, $x_l \in X_l$, $x_h \in X_h$ refers to the input function $p(\mathbf{r}, t)$ while $y_l \in Y_l$, $y_h \in Y_h$ refers to the output function $T(\mathbf{r}, t)$. Inspired by the latest work in multi-fidelity learning with tractable likelihood [9], we adopt an autoregressive structure:

$$F^h(x_h) = F^l(x_h) + F^r(x_h), \quad (6)$$

where F^h represents the target high-fidelity function. F^l is the low-fidelity function and F^r is the residual function, both of which is approximated by two separate FNO models. We name the framework for determining F^h as Autoregressive Operator Learning (ARO).

Fig. 3 shows the general framework of ARO, we first train F^l with $\mathcal{D}_l$, then train F^r with the differences Y_r , which is obtained by subtracting Y_h and the prediction $F^l(X_h)$ from trained F^l with X_h . Remember that Y_h and $F^l(X_h)$ are collections of functions defined in the same space due to the resolution-invariance of FNO, so the subtraction is always feasible. For inference, the unseen high-fidelity data X_h is input into the trained F^l and F^r models simultaneously while the predictive $\hat{Y}_h$ is obtained by adding the outputs of the two models together.

3.2 Uncertainty Quantification For ARO

In the domain of thermal analysis, grasping the range of temperature variations alongside their associated uncertainties proves immensely beneficial, especially in rectifying potential problems in chip design, which enables more precise and error-resilient design adjustments in thermal management systems for chips. In order to introduce uncertainty quantification in ARO, we employ the Monte Carlo dropout (MC-dropout) framework for our low- and high-fidelity FNO models.

MC-dropout was first put forward by [15], which demonstrated that the use of dropout (and its variants) in neural networks can be interpreted as a Bayesian approximation of the well-known probabilistic model, Gaussian Process (GP). Furthermore, it has given a complete theoretical treatment of the link between GP and dropout and examined uncertainty obtained by MC-dropout on the task of regression and classification.

For implementation, we maintain the original architecture of the ARO unchanged, randomly dropping ten percent of the connections after each neural network layer. A dropout rate of 10% implies that, during the forward propagation process of the model, each connection of every neuron has a 10% chance of being randomly deactivated. Dropout is applied during both the training and inference stages. Once the model is trained, running the model under the dropout mode is equivalent to making predictions using samples from the posterior distribution for the weight in the deep learning model, enabling us to quantify the confidence of the model prediction. Additionally, it enables the application of active learning strategies, which rely on accurate quantification of the model's weak regions, i.e., the areas where predictions are made with large uncertainty or large deviations.

3.3 Active learning

The multi-fidelity approach proves effective in lowering the computational expense involved in data preparation within ARO. Nevertheless, it requires a substantial amount of labeled data for training

and is thus computationally expensive. To further reduce the data preparation cost, we introduce active learning (AL) to guide the data preparation process and enables the model to be refined systematically with minimal labeled data. AL aims to optimize the data sampling process and expedite the training phase by learning an efficient scheme for data selection. Its primary objective is to strategically identify the most informative data points, thereby enhancing the learning process and minimizing the labeled data for model training. Li *et al.* [12] introduced a multi-fidelity active learning method named multi-resolution active learning (MRA) based on mutual information. MRA efficiently computes mutual information by exploiting the covariance matrix structure and employing matrix determinant lemma, which addresses concerns regarding computational overhead. Inspired by [12], we use the following equation as an acquisition function to evaluate the mutual information of two predictions, y_1 and y_2 :

$$\mathbb{I}(y_1, y_2 | \mathcal{D}) = \mathbb{H}(y_1 | \mathcal{D}) + \mathbb{H}(y_2 | \mathcal{D}) - \mathbb{H}(y_1, y_2 | \mathcal{D}). \quad (7)$$

$\mathbb{H}(\cdot)$ is the closed form of mutual information calculation function. Assuming multi-fidelity dataset $\mathcal{D}$ consists of low-fidelity dataset denoted as $\mathcal{D}_l = \{X_l, Y_l\}$, and high-fidelity dataset as $\mathcal{D}_h = \{X_h, Y_h\}$. Representing the prediction of ARO for low-fidelity by $\hat{y}_l$, prediction of high-fidelity by $\hat{y}_h$, together as $\hat{y} = [\hat{y}_l; \hat{y}_h]$, we first consider to calculate the last term of Eq. (7):

$$\mathbb{H}(\hat{y}) = \frac{1}{2} \log(\det[\text{Cov}(\hat{y} | \mathcal{D})]) + \text{const}. \quad (8)$$

By employing a Monte-Carlo approximation and matrix determinant lemma, we can get the following calculation,

$$\begin{aligned} \log(\det[\text{Cov}(\hat{y} | \mathcal{D})]) &= \log(\det[\Lambda + \mathbf{B}\mathbf{B}^T]) \\ &= \log(\det[\Lambda]) + \log(\det[\mathbf{I} + \mathbf{B}^T\Lambda^{-1}\mathbf{B}]). \end{aligned} \quad (9)$$

Among them, $\mathbf{I}$ is the identity matrix. Assuming M is the times of repeating prediction, then we can obtain an ensemble of the point estimates of the model parameters, $\{\theta_1^*, \dots, \theta_M^*\}$. Λ is a diagonal matrix expressed as:

$$\Lambda = \text{diag}\left(\frac{1}{M} \sum_{m=1}^M \sigma_{\theta_j^*}^2(x_l) \cdot \mathbf{I}, \frac{1}{M} \sum_{m=1}^M \sigma_{\theta_j^*}^2(x_h) \cdot \mathbf{I}\right), \quad (10)$$

$$\mathbf{B} = \frac{1}{\sqrt{M-1}} [\rho_1 - \mathbb{E}(\hat{y} | \mathcal{D}), \dots, \rho_M - \mathbb{E}(\hat{y} | \mathcal{D})],$$

Especially, $\mathbb{E}(\hat{y} | \mathcal{D})$ is calculated by

$$\mathbb{E}(\hat{y} | \mathcal{D}) = \frac{1}{M} \sum_{m=1}^M \rho_m, \quad (11)$$

where $\rho_m = [\mu_{\theta_m^*}(x_l); \mu_{\theta_m^*}(x_h)]$. Additionally, it is straightforward to use a similar method to compute $\mathbb{H}(y_1 | \mathcal{D})$ and $\mathbb{H}(y_2 | \mathcal{D})$.

By employing acquisition function $\mathbb{I}(\cdot)$, we can get the mutual information of each x_l and x_h . Mutual information becomes particularly crucial when we've depleted our limited dataset to train ARO but seek to generate additional data to further refine its accuracy. Typically, we can efficiently generate various power maps x_l and x_h , but obtaining the corresponding thermal distributions y_l and y_h is very time-consuming, often taking more than hundred times longer. Hence, employing strategies such as mutual information to assess the values of x_l and x_h for training, followed by promptly obtaining the corresponding y_l and y_h , would greatly benefit model training. Additionally, employing this strategy can guide the fusion

of low- to high-fidelity ratios. This is crucial, as adhering to training models at a uniform ratio may sometimes limit model performance.

We detail the specific active learning strategy in Algorithm 1. Initially, we randomly select k samples of each fidelity from the data that are not used in training to construct the candidate pool $\mathcal{P}$. Subsequently, we utilize the trained ARO model to get the predictions of samples in $\mathcal{P}$. Then, we apply $\mathbb{I}(\cdot)$ to compute the mutual information values (referred as MI). We arrange the samples in descending order based on their MI and select the top samples within cost budget C , along with their corresponding labels to join the subsequent training. Specifically, a balancing factor λ is introduced when compute the MI of high-fidelity samples. That is because the difference in MI of different fidelity samples in the early stages is minimal. If directly dividing the MI by corresponding time cost of each fidelity to reflect the cost difference, it may result in excessive penalization of high-fidelity samples. λ is set to gradually adjust to the real-time cost ratio (the time taken to generate y_l relative to the time taken to generate y_h) during training.

Algorithm 1 Active Learning for ARO

Input: Original multi-fidelity training dataset $\mathcal{D}$, trained ARO, active learning times T , cost budget C , candidate size k

Output: Optimized ARO

```

1: for  $t = 1$  to  $T$  do
2:   Randomly select  $k$  untrained samples of each fidelity as  $\mathcal{P}$ 
3:   Get predictions  $Y'$  for samples in  $\mathcal{P}$  using ARO
4:   for  $y$  in  $Y'$  do
5:      $MI(y) = \mathbb{I}(y, Y' | \mathcal{P})$ 
6:   end for
7:   Sort samples in  $\mathcal{P}$  by  $MI$  in descending order
8:    $X \leftarrow$  top samples from sorted  $\mathcal{P}$  without exceeding  $C$ 
9:    $\mathcal{D} \leftarrow \mathcal{D} \cup X$ 
10:  Retrain ARO on  $\mathcal{D}$ 
11: end for
```

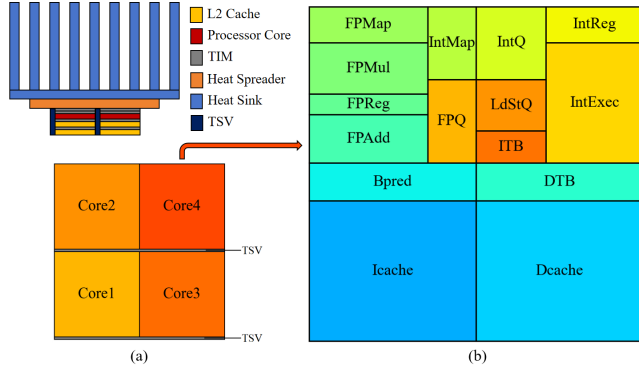
4 EXPERIMENTAL RESULTS

4.1 Experiment Setup

We chose three different 3D-ICs for thermal evaluations, their sizes are detailed in Table 1. Chip1 is a quad-core architecture implemented by three device layers. The core used is alpha 21264 ev6 microprocessor [16], which is located on the upper layer close to heatsink. L2 cache is located on the other two device layers. The address and data bus between L2 cache and processor core are connected by TSVs [17]. The detailed 3D stacking structure is shown in Fig. 4. Chip2 is a single core alpha 21264 ev6 microprocessor with 2 layers. It has similar 3D structure as Chip1. One layer has the processor core and L1 cache, and the other layer is L2 cache. Chip3 is an eight-core microprocessor with the similar architecture as Chip2 with 2 stacking device layers. One layer has four L2 cache banks, and the other layer has eight cores and L1 cache. MTA simulator [18] was used in thermal simulations to generate the training data because of its fast running speed and high accuracy compared to vendor tools (e.g., COMSOL). The data generation was run on a server with Intel Xeon CPU with 128GB memory. Our training model was constructed with PyTorch framework and ran on a NVIDIA A100 for model training.

Table 1: The geometric structures (mm) of three 3D ICs used as our test cases.

	Chip1	Chip2	Chip3
Device Layer	12.4×12.76×0.15	16×16×0.15	10×10×0.052
TIM Layer	12.4×12.76×0.02	16×16×0.02	10×10×0.02
TSV	diameter: 0.01; pitch: 0.01		
Heat Spreader	30×30×1.0		
Heat Sink	60×60×6.9		

**Figure 4: Structure and Floorplan of Chip1. (a) The stacking structure of Chip1 with four processing cores. (b) Detailed layout of one core.**

Generating power maps as input. We obtained the power trace of each functional block using Synopsys Power Compiler. Based on these traces, we generated more power profiles as our training data. In order to make reasonable power distributions, we generated power trace of each functional block within the same range as those obtained by Power Compiler. As for transient analysis, the power trace was collected for 9ms with a fixed time step $\Delta t = 1ms$.

Obtaining temperature maps as output. We utilized MTA to obtain temperature distributions. In our experiments, each dataset consisted of power and temperature distributions of the 3D-IC with both low-fidelity and high-fidelity grids. The simulation time ratios of low-fidelity and high-fidelity grids are shown in Table 2, which presents the runtime ratios of both steady and transient thermal simulations. The runtime corresponding to high-fidelity simulation is listed in the last column in this table. We considered the entire core of Chip3 as a single functional block to save thermal simulation time, while each component of the core in Chip1 and Chip2 was considered as individual functional blocks. Therefore Chip3 has a lower grid resolution compared to Chip1 and Chip2. Additionally, we took the time required to generate a low-fidelity training dataset as one unit of the runtime cost, and for convenience, we uniformly consider the low- to high-fidelity data time cost ratio to be 1:4 in the following experimental results.

ARO Settings. In the FNO, the mode parameter determines the frequency space utilized during the Fourier transform of the specific dimension. Setting the mode number lower implies that it captures lower frequency modes while truncating higher frequency modes. For steady-state analysis and transient thermal analysis, which involve three-dimensional and four-dimensional spaces respectively, the corresponding number of modes is three and four, respectively. Specifically, in our experiment, for steady-state datasets like Chip1-Steady, the mode of each FNO was set to [24, 24, 2] with a width of

Table 2: Resolution and runtime cost ratio for each 3D-IC under multi-fidelity grids.

Dataset	Resolution	Time Cost Ratio	Runtime on High-fidelity Grid (s)
Chip1-Steady	85× 85,	1: 4.805	479.1
Chip1-Transient	151× 151	1: 4.958	1002.5
Chip2-Steady	55× 55,	1: 4.465	236.2
Chip2-Transient	87× 87	1: 4.518	461.3
Chip3-Steady	39× 39,	1: 3.818	136.6
Chip3-Transient	63× 63	1: 3.910	227.5

* Runtime is averaged by multiple thermal simulations to eliminate time fluctuations.

64. However, for smaller-scale datasets such as Chip2-Steady and Chip3-Steady, we set the mode to [12, 12, 2] with a width of 32. As transient datasets include an additional time dimension, the mode for Chip1-Transient was [24, 24, 2, 2], while for Chip2-Transient and Chip3-Transient, it was [12, 12, 2, 2]. All FNOs referred in the experiments were composed of four Fourier layers.

Training and Testing. We trained ARO by 800 epochs with a batch size of 20 to guarantee convergence. Meanwhile, we set Adam optimizer with a learning rate of 10^{-3} and weight decay of 10^{-4} . Learning rate decay was scheduled every 100 epochs with a reduction factor of 0.5. L2 norm loss was utilized for optimization. The testing samples used in experiments were untrained samples reserved from the overall samples, sizes of which were kept as 500.

Indicators. To assess the performance of ARO, we considered Root Mean Square Error (RMSE) due to its general application in regression problems. We also introduced the R-squared (R^2) metric, which ranges from 0 to 1, with higher values indicating a superior ability to explain the variance of observed data. Besides, we considered the average temperature error (Avg.Terr) and maximum temperature error (Max.Terr) as indicators to assess our models. Mean absolute percentage errors (MAPEs) and Peak absolute percentage errors (PAPEs) were also included in experiments. These metrics together provided a comprehensive assessment of the model’s performance.

4.2 Comparisons to Single-fidelity Approaches

To demonstrate the effectiveness of the multi-fidelity fusion approach and the rationality of the ARO architecture, we compared ARO with the single-fidelity FNO models under the same time cost. As mentioned in Section 4.1, we considered the ratio of time spent in generating low- to high-fidelity data as 1:4. In other words, the time required to generate one high-fidelity sample can generate four low-fidelity samples.

Experiments were conducted on six datasets, and the outcomes are illustrated in Fig. 5. In this context, FNO_low refers to the model trained exclusively on low-fidelity data, while FNO_high denotes the model trained solely on high-fidelity data. ARO represents the model trained using multi-fidelity fusion data. To discern the influence of various levels of fidelity fusion, we established three fidelity fusion schemes with ratios of 2:1, 3:1, and 4:1, denoting the ratio of low- to high-fidelity data samples. The experimental results indicate that fusion schemes with these three ratios are significantly more effective than the single-fidelity trained model. In fact, if we continue to increase the amounts of data in each experiment of

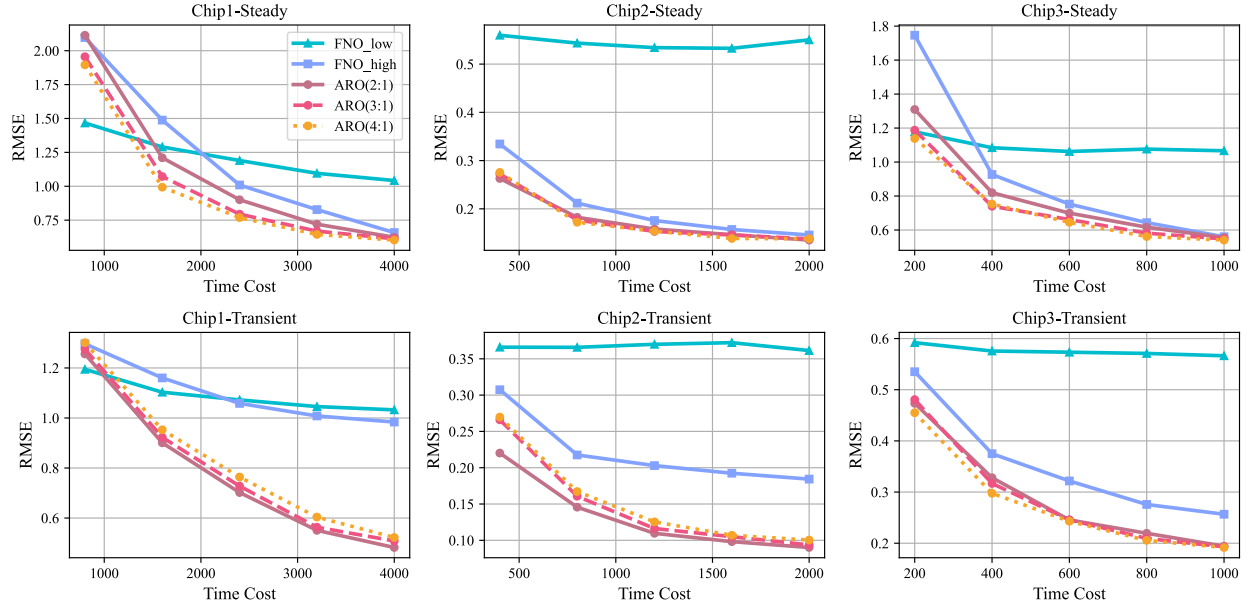


Figure 5: RMSE performance comparisons of single-fidelity FNO and ARO on three 3D-ICs.

Table 3: The detailed performances of AROs on three 3D-ICs.

Design	Time Cost	#Low-#High		RMSE	R ²	Mean.Terr(K)	Time(s)	Improvement(%) [*]
Chip1	4000	1333-666	Steady	0.6125	0.9846	0.3760	14137	6.9
			Transient	0.4823	0.9824	0.2374	12580	50.9
Chip2	2000	666-333	Steady	0.1352	0.9994	0.0643	4917	7.2
			Transient	0.09370	0.9973	0.0451	5476	51.1
Chip3	1000	332-166	Steady	0.5407	0.9962	0.0643	2441	3.4
			Transient	0.1929	0.9963	0.1444	2667	20.0

^{*} The improvement refers to RMSE improvement compared to high-fidelity FNOs.

Fig. 5, AROs and single-fidelity FNOs will demonstrate improved performance. Additionally, RMSE trend reveals that different ratios may achieve superior performance at different data costs.

For detailed presentation, Table 3 provides the specific performance of AROs under training configuration corresponding to last point of each experiment in Fig. 5. The differences in training sample settings among the three designs stems from the varying grid sizes (Chip1 having the most complex grid, while Chip3 possesses the simplest grid), thereby the data requirements and predicting performance with each model differs. Moreover, we continued the experiment on the three steady-state datasets in Fig. 5, observing that when the training cost increased to 5400, 2600, and 1400 respectively, the FNO_high model achieved accuracy comparable to ARO with a training cost of 4000, 2000, and 1000. So ARO can save thermal simulation time by 25%, 23%, and 28% respectively. In the corresponding transient experiments, as depicted in the Table 3, the enhancements of AROs were more pronounced, leading to a greater reduction in time cost. Besides, we present the prediction outcomes of ARO on Chip2 in Fig. 6 and Fig. 7 to illustrate the effectiveness of our proposed approach.

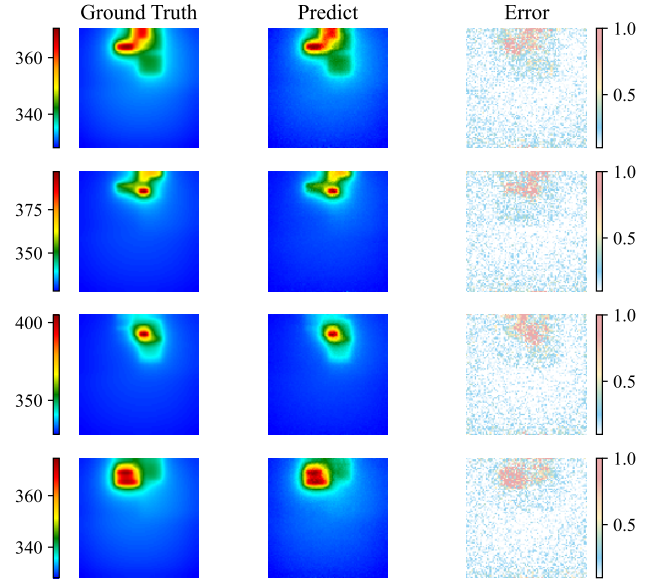


Figure 6: Comparisons between ARO predictions and MTA-generated thermal distributions on Chip2-Steady.

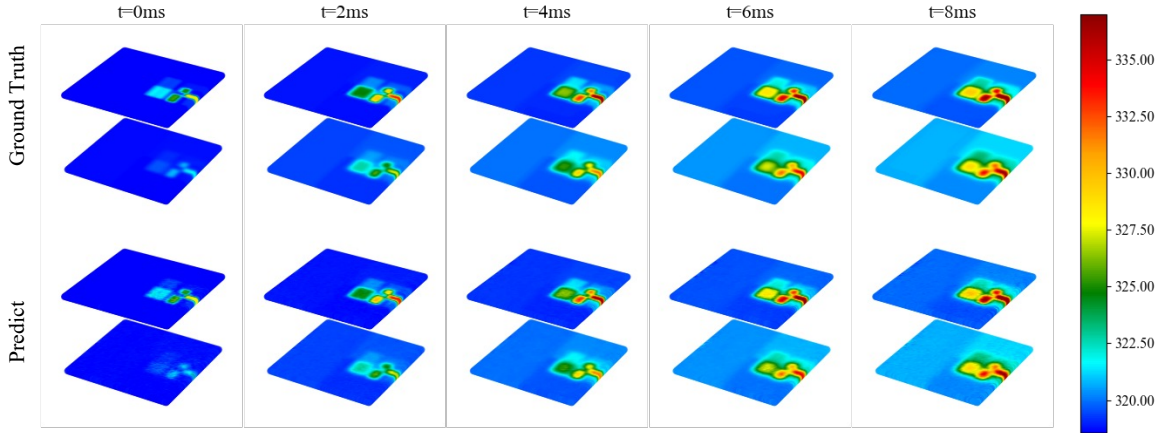


Figure 7: ARO predictions and MTA-generated thermal distributions with time steps on Chip2-Transient.

4.3 Comparisons to SOTA ML-Baselines

In recent years, the application of machine learning methods to thermal analysis has seen a surge, leading to the emergence of various approaches such as Thermal-PINN and Thermal-GAN. These methods offer distinct methodologies and perspectives aimed at addressing the complexities of thermal analysis challenges. As discussed in the introduction, due to the disparate problem definitions inherent in these mainstream models, direct comparison cannot be made in this work. However, the model that aligns most closely with our problem definition is DeepOHeat, as demonstrated in [8], where power distribution and other constraints serve as inputs to train DeepONet operators. In order to assess the progress made by our proposed ARO model, we considered DeepOHeat as one of the baselines. This comparison aims to highlight the effectiveness of our multi-fidelity fusion method. Additionally, we compared our approach with GAR, a state-of-the-art framework that utilizes multi-fidelity fusion techniques. By comparing with GAR, we intend to emphasize the inherent advantages and potential of FNO within the domain of multi-fidelity fusion.

All comparative experiments were performed on the Chip2-Steady dataset, as both baseline models lack the capability to handle four-dimensional data, thus precluding a comparison with transient datasets. Since the DeepOHeat model was not open source yet, we replicated the DeepONet-based DeepOHeat according to the method description provided in [8]. As described in [8], we employed a 9-layer branch net with 256 neurons per layer combined with a 6-layer trunk net with 128 neurons per layer. Given that DeepOHeat is not a multi-fidelity model, we utilized high-fidelity samples of 200, 400 respectively, while ensuring that the time cost remained consistent (GAR and ARO both utilized 400 high-fidelity and 100 low-fidelity samples, and 800 high-fidelity and 200 low-fidelity samples, respectively, as they are multi-fidelity models). However, the actual predictive performance obtained using DeepOHeat were not satisfactory. The average temperature error of the resulting temperature distribution exceeded 2K, which significantly deviated from the predictive capabilities of the other two models. Hence, we mainly present the predictive performance of ARO and GAR in Table 4. The ineffectiveness of DeepOHeat may stem from its inability to effectively learn from small-size, higher-resolution

Table 4: Comparisons of ARO and SOTA on Chip2-Steady.

#Low-#High		Max.Terr(K)	Avg.Terr(K)	MAPE(%)	PAPE(%)
400-100	GAR	7.656	0.1848	0.1238	1.9834
	ARO	2.2524	0.0795	0.0233	0.6166
800-200	GAR	5.7168	0.0755	0.1000	1.5101
	ARO	1.8533	0.0660	0.0194	0.5044

samples (considering 21x21 resolution used in DeepOHeat experiments). From Table 4, it’s evident that ARO outperforms GAR significantly in terms of both average temperature differences and maximum temperature differences.

4.4 Comparisons to Thermal Simulation Tools

To evaluate the efficiency of ARO for thermal simulation, we compared the speed and accuracy of ARO with thermal simulation tools like MTA [18], COMSOL [19], and Hotspot [20] on Chip2. Specifically, We further generated 50 sets of power maps for both steady-state and transient experiments respectively following the method outlined in Section 4.1. To obtain the simulation results of the mentioned tools, we configured MTA with the finest grid, resulting in a number of degree of freedom (DOF) of 589,311. Similarly, we set the grid in COMSOL to the finest resolution to ensure the most accurate results. For Hotspot, we used a 64x64 grid on each layer, while ARO utilized an 85x85 grid on each layer, consistent with the highest resolution used during training.

We obtained the maximum surface temperatures of each device layer in 50 sets of steady-state and transient thermal simulations using various tools. The distributions are illustrated in Fig. 8. It can be observed that the distributions of ARO’s predictions of the maximum temperatures closely resemble that of COMSOL and MTA. For a more detailed presentation, we provide the specific values of the maximum and minimum temperatures of five steady-state samples (referred to as p_1, p_2, p_3, p_4, p_5) in Table 5. Compared to COMSOL, ARO estimates the maximum temperature within 0.3K in both steady-state and transient analysis, demonstrating its accuracy in thermal simulations.

Since Hotspot and MTA are simulation tools known for their faster performance compared to COMSOL, here we mainly compare the speedup ratio of ARO with Hotspot and MTA as displayed in Table 6. The time values represent the average time taken by ARO,

MTA, and Hotspot to complete 50 sets of steady-state and transient thermal simulations. ARO achieved an impressive 999 $\times$ speedup compared to MTA and a 473 $\times$ speedup compared to Hotspot for steady-state analysis. Additionally, for transient analysis, ARO achieved 1213 $\times$ speedup and 296 $\times$ speedup respectively.

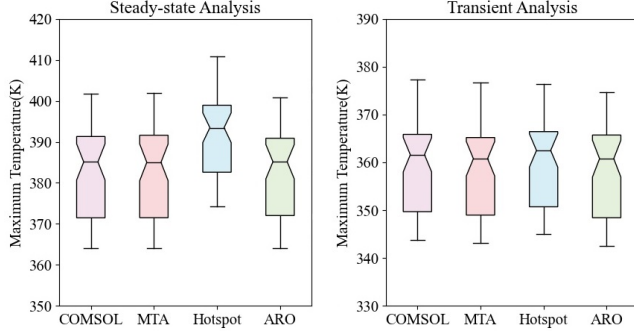


Figure 8: Temperature comparisons among COMSOL, MTA, Hotspot and ARO on Chip2.

Table 5: Maximum and minimum temperature comparisons among COMSOL, MTA, Hotspot and ARO on five steady-state samples.

		COMSOL	MTA	Hotspot	ARO	Error [*]
p_1	Max(K)	389.730	389.583	398.950	389.689	-0.041
	Min(K)	328.073	328.115	334.950	328.060	+0.017
p_2	Max(K)	398.464	398.351	400.380	398.457	-0.007
	Min(K)	328.043	328.085	334.920	327.767	-0.276
p_3	Max(K)	368.420	368.513	382.040	368.479	+0.059
	Min(K)	328.155	328.197	335.170	328.146	-0.009
p_4	Max(K)	364.546	364.525	374.290	364.592	+0.046
	Min(K)	328.133	328.175	335.150	328.140	+0.007
p_5	Max(K)	371.083	371.087	379.370	371.070	-0.013
	Min(K)	328.045	328.087	334.910	328.038	-0.007

^{*} Error refers to the difference between ARO and COMSOL.

Table 6: Comparisons of runtime among ARO, MTA and Hotspot.

Analysis		ARO	MTA	Hotspot
Steady	Runtime(s)	0.23	229.86	108.96
	Speedup	1 $\times$	999$\times$	473$\times$
Transient	Runtime(s)	0.38	461.10	112.84
	Speedup [*]	1 $\times$	1213$\times$	296$\times$

^{*} Speedup refers to the ratio of the simulation tool's runtime versus ARO's runtime.

4.5 Validation of Active Learning

As shown in Fig. 5, as the performance of ARO improves, continuing to train the model with a fixed ratio could restrict its capabilities. At this point, active learning strategies become essential to guide us in data preparation and determining the ratio of low- to high-fidelity. By inputting multiple multi-fidelity samples into the trained ARO and employing an acquisition function, we can calculate the mutual information of each sample. This enables us to determine which x_l and x_h samples to add to the training set in the next stage, as well as how many data samples of each-fidelity to choose.

To validate the efficacy of the proposed active learning algorithm, we conducted ablation studies on Chip2-Steady and Chip3-Steady. We opted not to conduct experiments on the transient dataset due to its numerous time steps (up to 9), which complicates the calculation of mutual information. In each experiment, we initially trained an ARO model with low- to high-fidelity data number being 400-100. Next, we assessed the impact of augmenting the training data in various ways: randomly selecting high-fidelity data, adjusting the low-fidelity to high-fidelity ratio to 4:1, 3:1, 2:1 (also chosen randomly), and employing active learning to select multi-fidelity data for training. Throughout these experiments, we maintained consistent data cost, with a cost budget of 100 added each time. We set k to 200 and λ to the difference in average mutual information values between high-fidelity and low-fidelity samples. The specific experimental results are depicted in Fig. 9. It is evident from the experimental results that, in comparison with manually setting the training ratios and simply augmenting with high-fidelity data, the incorporation of valuable low-fidelity and high-fidelity data selected by active learning strategy can more effectively enhance the overall prediction performance of ARO. Additionally, for Chip2-Steady, achieving the accuracy of ARO with a 1200 time cost requires 1600 time cost for randomly selected high-fidelity training. Similarly, for Chip3-Steady, attaining the accuracy of ARO with a 1000 time cost demands more than 1600 time cost for randomly selected high-fidelity training. This analysis suggests that active learning can streamline data preparation efforts, resulting in a reduction of data preparation time cost by approximately 25% and 27.5%, respectively.

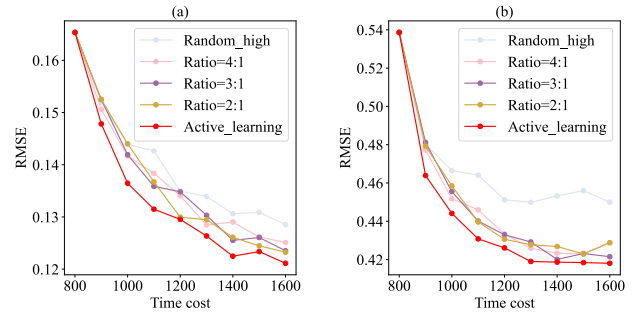


Figure 9: RMSE performance of active learning and other training data selection strategies on ARO. (a) Performance on Chip2-Steady, (b) Performance on Chip3-Steady.

5 CONCLUSION

We propose a novel ML-based approach, ARO, for predicting the thermal behavior of 3D integrated circuits by combining SOTA ML methods—operator learning and multi-fidelity fusion. ARO consistently outperforms the baseline with significantly lower computational costs. In scenarios with limited samples, the average prediction error for steady-state thermal analysis is less than 0.4K. Besides, ARO can simultaneously predict transient temperature changes at nine different time points, with an average error less than 0.3K. Additionally, the ARO configuration, incorporating the proposed active learning algorithm, achieves a minimum of 25% training data reduction. Our future work will focus on the application of ARO to other thermal analysis tasks, such as thermal-aware placement and routing, and the development of a more efficient active learning algorithm for transient thermal analysis.

REFERENCES

- [1] Peng Li, Lawrence T Pileggi, Mehdi Asheghi, and Rajit Chandra. Efficient full-chip thermal modeling and analysis. In *IEEE/ACM International Conference on Computer Aided Design, 2004. ICCAD-2004.*, pages 319–326. IEEE, 2004.
- [2] Jai-Ming Lin, Wei-Yi Chang, Hao-Yuan Hsieh, Ya-Ting Shyu, Yeong-Jar Chang, and Juin-Ming Lu. Thermal-aware floorplanning and tsv-planning for mixed-type modules in a fixed-outline 3-d ic. *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, 29(9):1652–1664, 2021.
- [3] Hameedah Sultan, Anjali Chauhan, and Smruti R Sarangi. A survey of chip-level thermal simulators. *ACM Computing Surveys (CSUR)*, 52(2):1–35, 2019.
- [4] Arvind Sridhar, Alessandro Vincenzi, Martino Ruggiero, Thomas Brunschweiler, and David Atienza. 3d-ice: Fast compact transient thermal modeling for 3d ics with inter-tier liquid cooling. In *2010 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, pages 463–470, 2010.
- [5] Liang Chen, Jincong Lu, Wentian Jin, and Sheldon X.-D. Tan. Fast full-chip parametric thermal analysis based on enhanced physics enforced neural networks. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 1–8, 2023.
- [6] Wentian Jin, Sherif Sadiqbatcha, Jinwei Zhang, and Sheldon X.-D. Tan. Full-chip thermal map estimation for commercial multi-core cpus with generative adversarial learning. In *2020 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*, pages 1–9, 2020.
- [7] Jincong Lu, Jinwei Zhang, and Sheldon X.-D. Tan. Real-time thermal map estimation for amd multi-core cpus using transformer. In *2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, pages 1–7, 2023.
- [8] Ziyue Liu, Yixing Li, Jing Hu, Xinling Yu, Shinyu Shiao, Xin Ai, Zhiyu Zeng, and Zheng Zhang. Deepoheat: Operator learning-based ultra-fast thermal simulation in 3d-ic design. In *2023 60th ACM/IEEE Design Automation Conference (DAC)*, pages 1–6, 2023.
- [9] Yuxin Wang, Zheng Xing, and Wei W. Xing. Gar: generalized autoregression for multi-fidelity fusion. In *Proceedings of the 36th International Conference on Neural Information Processing Systems, NIPS '22*. Curran Associates Inc., 2024.
- [10] Yunyang Zhang, Zhiqiang Gong, Weien Zhou, Xiaoyu Zhao, Xiaohu Zheng, and Wen Yao. Multi-fidelity surrogate modeling for temperature field prediction using deep convolution neural network. *Engineering Applications of Artificial Intelligence*, 123:106354, 2023.
- [11] Yanfang Lyu, Xiaoyu Zhao, Zhiqiang Gong, Xiao Kang, and Wen Yao. Multi-fidelity prediction of fluid flow and temperature field based on transfer learning using fourier neural operator. *arXiv preprint arXiv:2304.06972*, 2023.
- [12] Shibo Li, Xin Yu, Wei Xing, Robert Kirby, Akil Narayan, and Shandian Zhe. Multi-resolution active learning of fourier neural operators. In *International Conference on Artificial Intelligence and Statistics*, pages 2440–2448. PMLR, 2024.
- [13] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. *arXiv preprint arXiv:2010.08895*, 2020.
- [14] Akshay Thakur, Tapas Tripura, and Souvik Chakraborty. Multi-fidelity wavelet neural operator with application to uncertainty quantification, 2023.
- [15] Yarin Gal and Zoubin Ghahramani. Dropout as a bayesian approximation: Representing model uncertainty in deep learning. In *international conference on machine learning*, pages 1050–1059. PMLR, 2016.
- [16] Richard E Kessler. The alpha 21264 microprocessor. *IEEE micro*, 19(2):24–36, 1999.
- [17] Zongqing Ren, Ayed Alqahtani, Nader Bagherzadeh, and Jaeho Lee. Thermal analysis of 3d ics with tsvs placement optimization. In *International Electronic Packaging Technical Conference and Exhibition*, volume 59322, page V001T01A006. American Society of Mechanical Engineers, 2019.
- [18] Scott Ladenheim, Yi-Chung Chen, Milan Mihajlović, and Vasilis F. Pavlidis. The mta: An advanced and versatile thermal simulator for integrated systems. *IEEE TCAD*, 37(12):3123–3136, 2018.
- [19] Multiphysics C. Introduction to comsol multiphysics. *COMSOL Multiphysics, Burlington, MA*, accessed Feb, 9(2018):32, 1998.
- [20] Wei Huang, Shougata Ghosh, Sivakumar Velusamy, Karthik Sankaranarayanan, Kevin Skadron, and Mircea R Stan. Hotspot: A compact thermal modeling methodology for early-stage vlsi design. *IEEE Transactions on very large scale integration (VLSI) systems*, 14(5):501–513, 2006.